

1. Plagiarism Detection - Phase 1 Report

Key Data Structures and Their Purpose:

- **struct Interval:** Stores details of an exact match (start, end, size, and indSec).
- **class IntervalTree:** Wrapper over `std::set` for non-overlapping Interval objects, ensuring sorted and conflict-free intervals.

Exact Short Sequence Detection:

`exact_short_match` detects short sequence matches (10–25 tokens) using `solve`. Matches are stored in `IntervalTree`.

- Iterates subsequences of lengths in range and compares using `solve`.
- Matches are stored ensuring no overlaps.

Function Signatures:

```
void exact_short_match(std::vector<int>& sub1, std::vector<int>& sub2,
                      int mini, int maxi, IntervalTree& t);
bool solve(std::vector<int>& sub1, std::vector<int>& sub2,
           int i, int j, int len, IntervalTree& t);
```

Longest Approximate Match Detection:

`longestApproxMatch` identifies the longest sequence with $\geq 90\%$ match from `IntervalTree`.

- Iterates stored intervals; allows up to 10% unmatched tokens using `accomodationPossible`.
- Tracks maximum match length and indices.

```
int longestApproxMatch(IntervalTree& t, int& fir, int& sec);
bool accomodationPossible(int unmatched, int matched);
```

Workflow:

1. **Entry Point:** `match_submissions` initiates `exact_short_match`, computes total match length, and identifies longest approximate matches.
2. **Flagging:** A submission is plagiarized if:
 - **Exact Matches:** Total match length $\geq 40\%$ of submission tokens.
 - **Approx Matches:** Longest match $\geq 30\%$ of submission tokens.
3. Returns: `{plagiarized, totalMatchLength, longestMatch, index_first, index_second}`.

Time and Space Complexity

Space: $O(k)$, where k is intervals in `IntervalTree`.

Time: $O(n \cdot m \cdot \Sigma(\text{len}))$, where n, m are submission sizes, and `len` is the sum of subsequence lengths (10–25).

2. Plagiarism Detection - Phase 2 Report

1. Detection of Long Pattern Matches

Uses DP to detect matches ≥ 75 tokens.

```
int longestExactMatch(const std::vector<int>& sub1,
                     const std::vector<int>& sub2, int& __numMatch,
                     std::shared_ptr<submission_t>& __currsubmission,
                     std::shared_ptr<submission_t>& __prevsubmission,
                     bool _sametime = false);
```

Time: $O(n \times m)$ per file pair. **Space:** $O(n \times m)$.

2. Detection of Patchwork Plagiarism

Tracks short matches (15 tokens) using a `std::set` for intervals. Flags plagiarism if ≥ 20 intervals.

```
void __addInterval(int s, int e,
                  std::shared_ptr<submission_t>& __currsession);
```

Time: $O(\log k)$ per insertion. **Space:** $O(k)$.

3. Detection of Short Pattern Matches

Short matches (15 tokens) are identified during DP traversal in `longestExactMatch`.

Threading and Concurrency Features

Implementation:

- **Thread:** `watcherThread` monitors `__uncheckedSubmissions`.
- **Synchronization:** `std::mutex` for shared resource access, `std::condition_variable` to signal thread.

Non-blocking Behavior: `add_submission` pushes submissions to a queue and signals the background thread.

```
void add_submission(std::shared_ptr<submission_t> __submission) {
    std::lock_guard<std::mutex> lock(this->mtx);
    //process
    __uncheckedSubmissions.push(__submission);
    cv.notify_one();
}
```

File Matching Based on Timestamp

The map `'__timestamp'` is used to track submission times. Files are compared against others submitted within one second of the current file. If the difference is less than one second, both files are flagged. Otherwise, only the later submission is flagged, on reaching the condition of plagiarism.

Helpers, Data Structures, and Flow

- **Data Structures:** `std::unordered_map` (timestamps, flagged statuses), `std::queue` (pending submissions), `std::set` (intervals for patchwork plagiarism).
- **Helpers:** `__tokenizer`, `__addInterval`, and `match_submissions`.

Flow:

1. Constructor initializes members, tokenizes past submissions, and starts `watcherThread`.
2. `add_submission` queues submissions and updates timestamps.
3. `__plagiarism_detection` processes submissions and flags plagiarism based on matches.